

Serverless Real-time Chat Application with Global Distribution

A Serverless Real-time Chat Application with Global Distribution is a chat app that allows users from all over the world to send and receive messages instantly, without you having to manage any servers.

What is a Serverless Real-time Chat Application?

- **Serverless:** You don't manage any servers. AWS handles the infrastructure, so you only focus on your code.
- **Real-time:** Messages are sent and received instantly, without delays.
- **Global Distribution:** The chat app can be accessed from anywhere in the world, with low latency

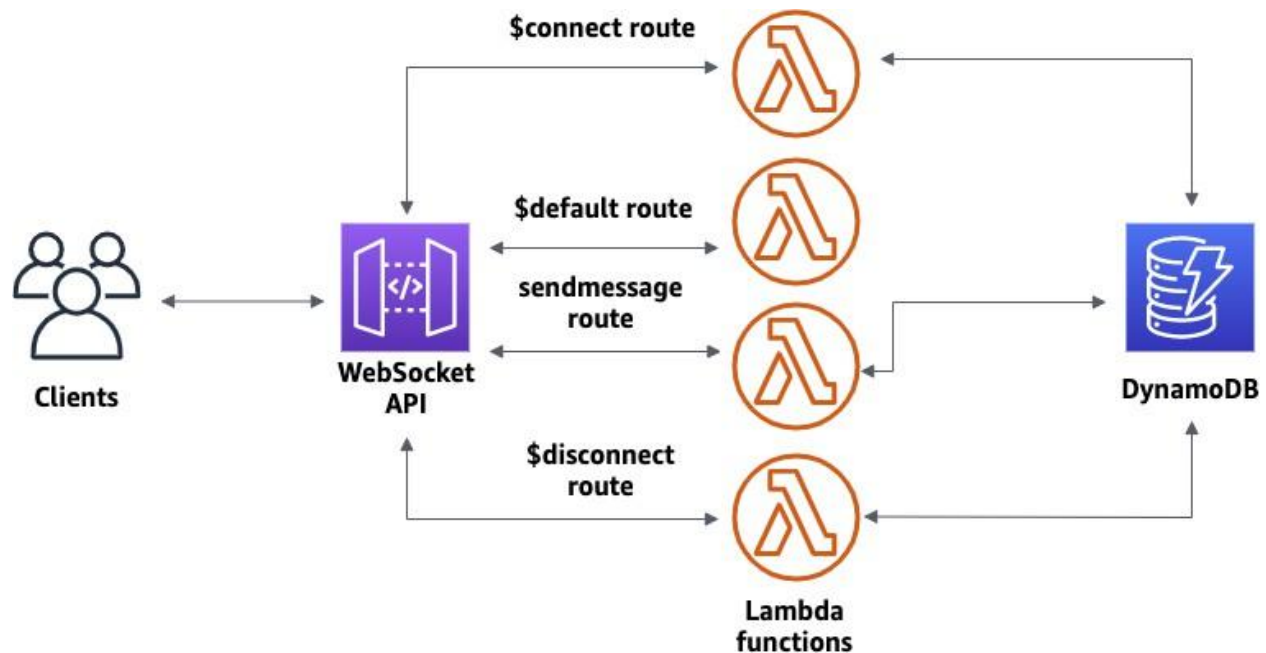
Prerequisites

- An AWS account
- Basic understanding of AWS Lambda, DynamoDB, and API Gateway
- AWS CLI and SAM CLI installed (optional, if you're using CloudFormation)

Key Components Involved

- **API Gateway (WebSocket):** Manages the connections between clients (chat users). It enables two-way communication.
- **AWS Lambda:** Small pieces of code (functions) that execute when something happens (like when a user sends a message).
- **Amazon DynamoDB:** A fast, NoSQL database to store data like user connections or chat messages.
- **Amazon CloudFront:** A Content Delivery Network (CDN) that speeds up access to your app by serving it from servers around the world.
- **Amazon S3:** Storage for your web app files (HTML, JavaScript, etc.), which CloudFront can distribute globally.

First, you'll use an AWS CloudFormation template to create Lambda functions that will handle API requests, as well as a DynamoDB table that stores your client IDs. Then, you'll use the API Gateway console to create a WebSocket API that integrates with your Lambda functions. Lastly, you'll test your API to verify that messages are sent and received.



Step 1: Create Lambda Functions and a DynamoDB Table

In this step, we will create the necessary backend components for your serverless chat application. These components include Lambda functions to manage client connections and a DynamoDB table to store each client's unique ID.

Download and Unzip the CloudFormation Template

<https://docs.aws.amazon.com/apigateway/latest/developerguide/samples/ws-chat-app-starter.zip>

1. Download the Template:

- Obtain the CloudFormation template file that will set up your DynamoDB table and Lambda functions. This file is usually provided in .yaml and .json format.

2. Unzip the Template:

- If the template is in a compressed format (e.g.zip), unzip it to access the .yaml or .json file.

Deploy the CloudFormation Stack

1. Open AWS Management Console:

- Go to the [AWS Management Console](#).

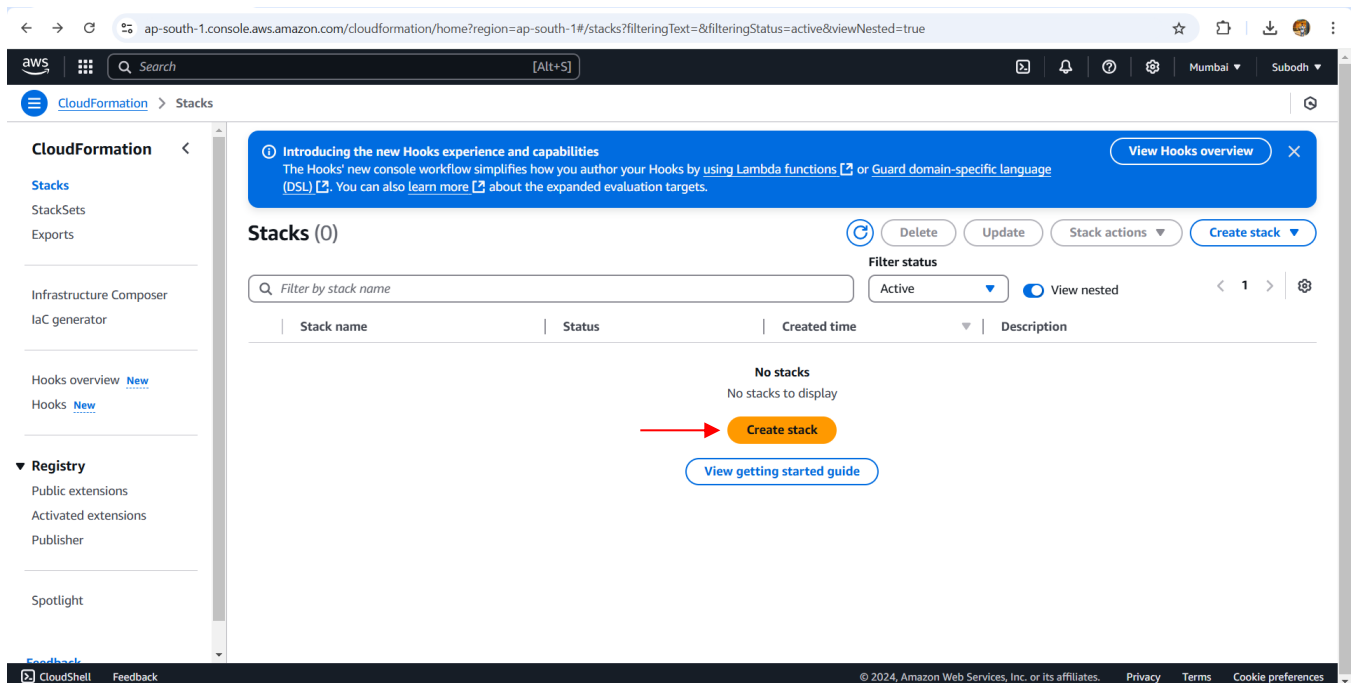
2. Navigate to CloudFormation:

- In the AWS Management Console, search for and select "CloudFormation".

3. Create a New Stack:

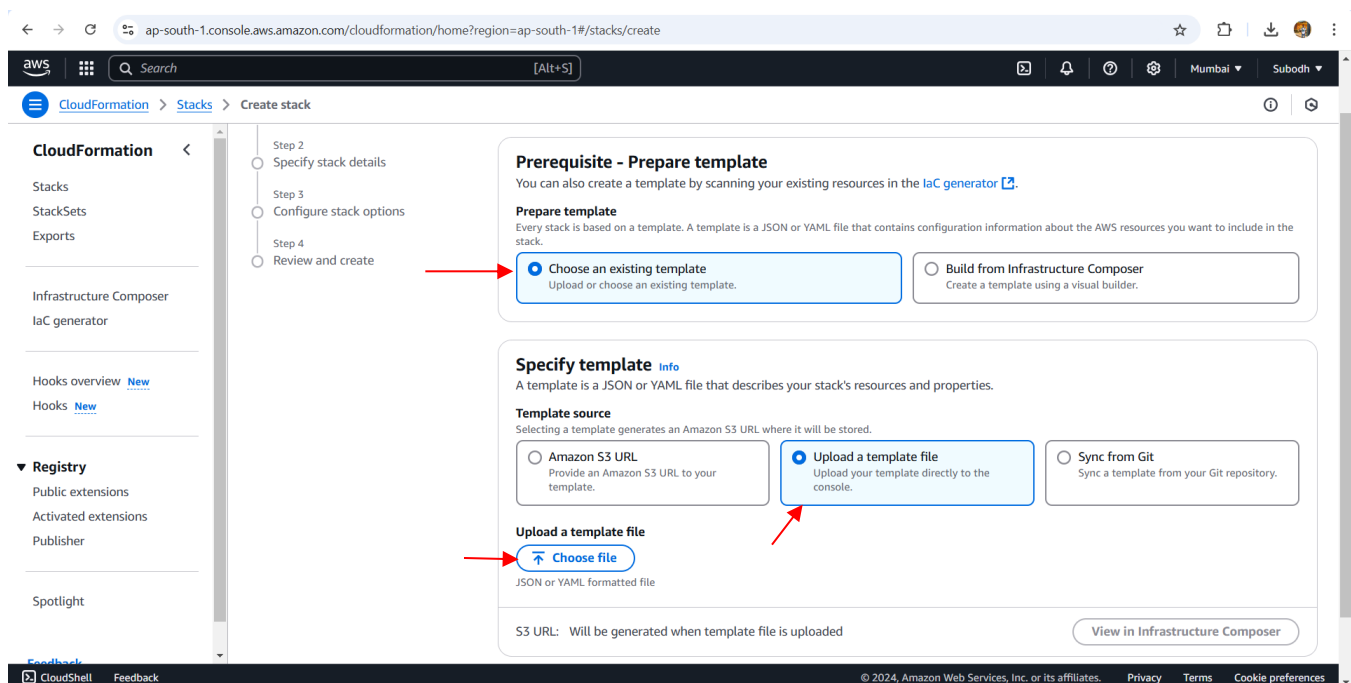
- Click on "**Create stack**" and then choose "**With new resources (standard)**".

Project by Subodh Kumar



4. Upload the Template File:

- In the "Specify template" section, choose "Upload a template file".



- Select the .yaml or .json file you unzipped earlier.
- Choose Next.

5. Configure the Stack:

- Give your stack a name, such as serverless-chat and then choose Next

The screenshot shows the AWS CloudFormation console in the 'Specify stack details' step. The left sidebar shows the 'Create stack' process with four steps: Step 1: Create stack, Step 2: Specify stack details (selected), Step 3: Configure stack options, and Step 4: Review and create. The main content area is titled 'Specify stack details' and contains two sections: 'Provide a stack name' and 'Parameters'. In the 'Provide a stack name' section, the 'Stack name' field is filled with 'serverless-chat'. A red arrow points to this field. Below the field, a note states: 'Stack name must be 1 to 128 characters, start with a letter, and only contain alphanumeric characters. Character count: 15/128.' The 'Parameters' section is titled 'Parameters' and contains the text 'No parameters' and 'There are no parameters defined in your template'. At the bottom right, there are three buttons: 'Cancel', 'Previous', and 'Next'. A red arrow points to the 'Next' button.

- For Configure stack options no changes, choose Next.

The screenshot shows the AWS CloudFormation console in the 'Configure stack options' step. The left sidebar shows the 'Create stack' process with four steps: Step 1: Create stack, Step 2: Specify stack details, Step 3: Configure stack options (selected), and Step 4: Review and create. The main content area is titled 'Configure stack options' and contains three sections: 'Tags - optional', 'Permissions - optional', and 'Stack failure options'. The 'Tags - optional' section has a description: 'Tags (key-value pairs) are used to apply metadata to AWS resources, which can help in organizing, identifying, and categorizing those resources. You can add up to 50 unique tags for each stack.' and a note: 'No tags associated with the stack.' Below this is an 'Add new tag' button and a note: 'You can add 50 more tag(s)'. The 'Permissions - optional' section has a description: 'Specify an existing AWS Identity and Access Management (IAM) service role that CloudFormation can assume.' and a sub-section 'IAM role - optional' with a note: 'Choose the IAM role for CloudFormation to use for all operations performed on the stack.' Below this is a form with 'IAM role name' and 'Sample-role-name' dropdowns, a 'Remove' button, and a refresh icon. The 'Stack failure options' section is titled 'Stack failure options' and has a sub-section 'Behavior on provisioning failure'.

Project by Subodh Kumar

- For Capabilities, acknowledge that AWS CloudFormation can create IAM resources in your account.
- Choose submit

The screenshot shows the 'Create stack' wizard in the AWS CloudFormation console. The 'Capabilities' section is highlighted, indicating that the template contains IAM resources. A red arrow points to the checkbox 'I acknowledge that AWS CloudFormation might create IAM resources', which is checked. Another red arrow points to the 'Next' button.

CloudFormation > **Stacks** > Create stack

Capabilities

The following resource(s) require capabilities: [AWS::IAM::Role]

This template contains Identity and Access Management (IAM) resources that might provide entities access to make changes to your AWS account. Check that you want to create each of these resources and that they have the minimum required permissions. [Learn more](#)

☒ I acknowledge that AWS CloudFormation might create IAM resources.

[Cancel](#) [Previous](#) [Next](#)

- AWS CloudFormation provisions the resources specified in the template. It can take a few minutes to finish provisioning your resources.

The screenshot shows the AWS CloudFormation console for the 'serverless-chat' stack. The stack is in the 'CREATE_IN_PROGRESS' state. The 'Events' tab is selected, showing a list of events. The first event is 'CREATE_IN_PROGRESS' with a timestamp of '2024-12-24 18:57:00 UTC+0530'.

CloudFormation > **Stacks** > serverless-chat

Stacks (1)

Filter by stack name

Filter status: Active View nested

Stacks

serverless-chat
2024-12-24 18:57:00 UTC+0530
CREATE_IN_PROGRESS

serverless-chat

[Delete](#) [Update](#) [Stack actions](#) [Create stack](#)

[Stack info](#) [Events - updated](#) [Resources](#) [Outputs](#) [Parameters](#) [Template](#) [Ch](#)

[Table view](#) [Timeline view - new](#)

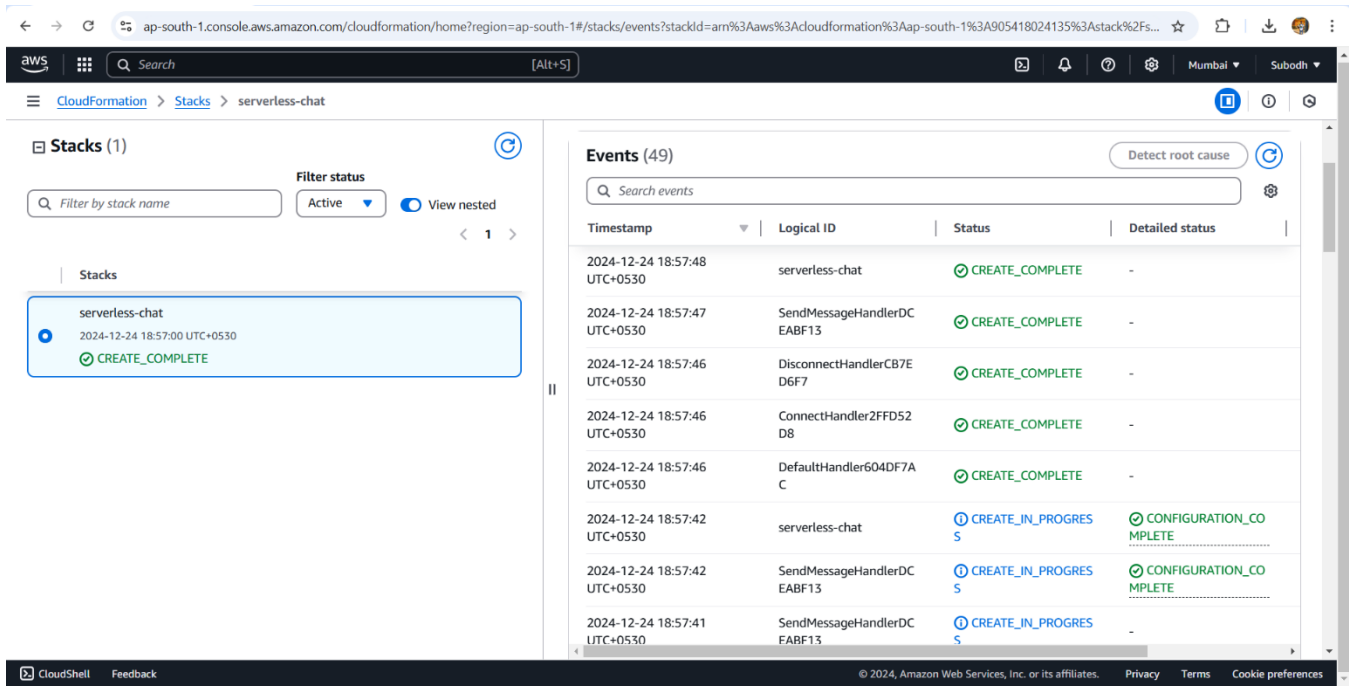
Events (1)

Detect root cause

Search events

Timestamp	Logical ID	Status	Detailed status	St
2024-12-24 18:57:00 UTC+0530	serverless-chat	CREATE_IN_PROGRES S	-	U

- When the status of your AWS CloudFormation stack is CREATE_COMPLETE, you're ready to move on to the next step.



Step 2: Create a WebSocket API

- You'll create a WebSocket API to handle client connections and route requests to the Lambda functions that you created in Step 1.

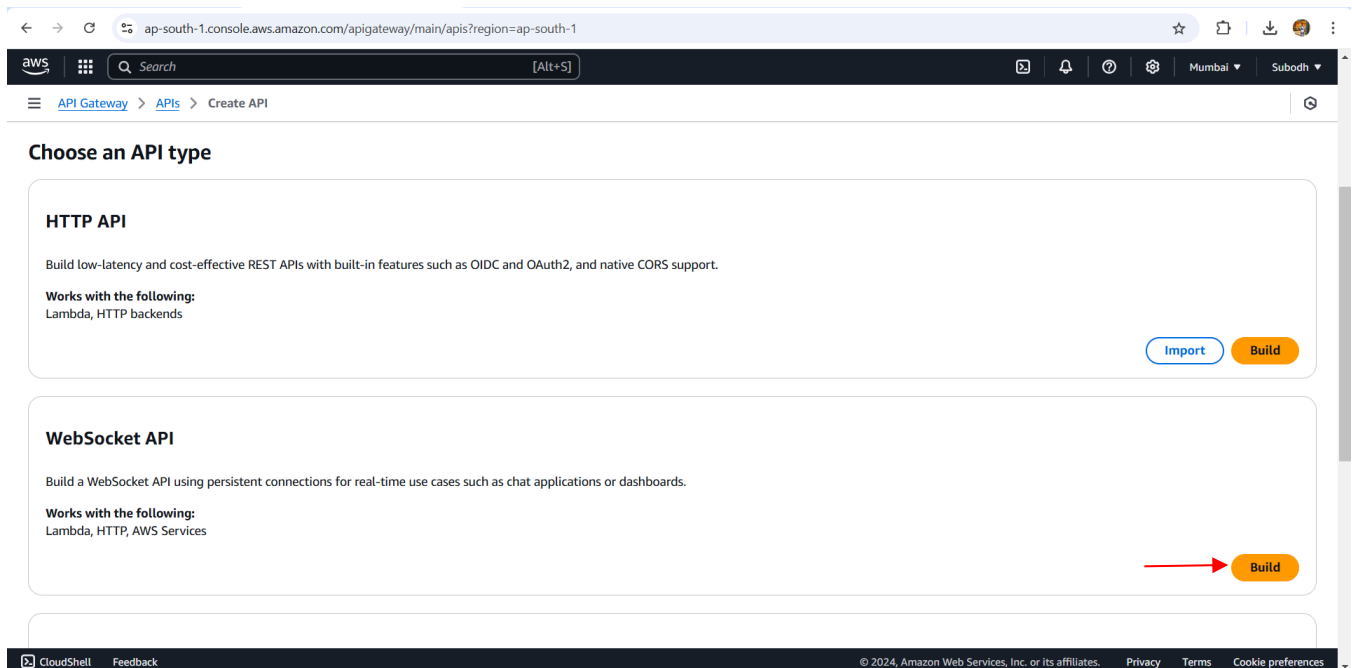
1. Sign In to the API Gateway Console

- Access API Gateway:
 - Go to the API Gateway console: API Gateway Console.

Create a New WebSocket API

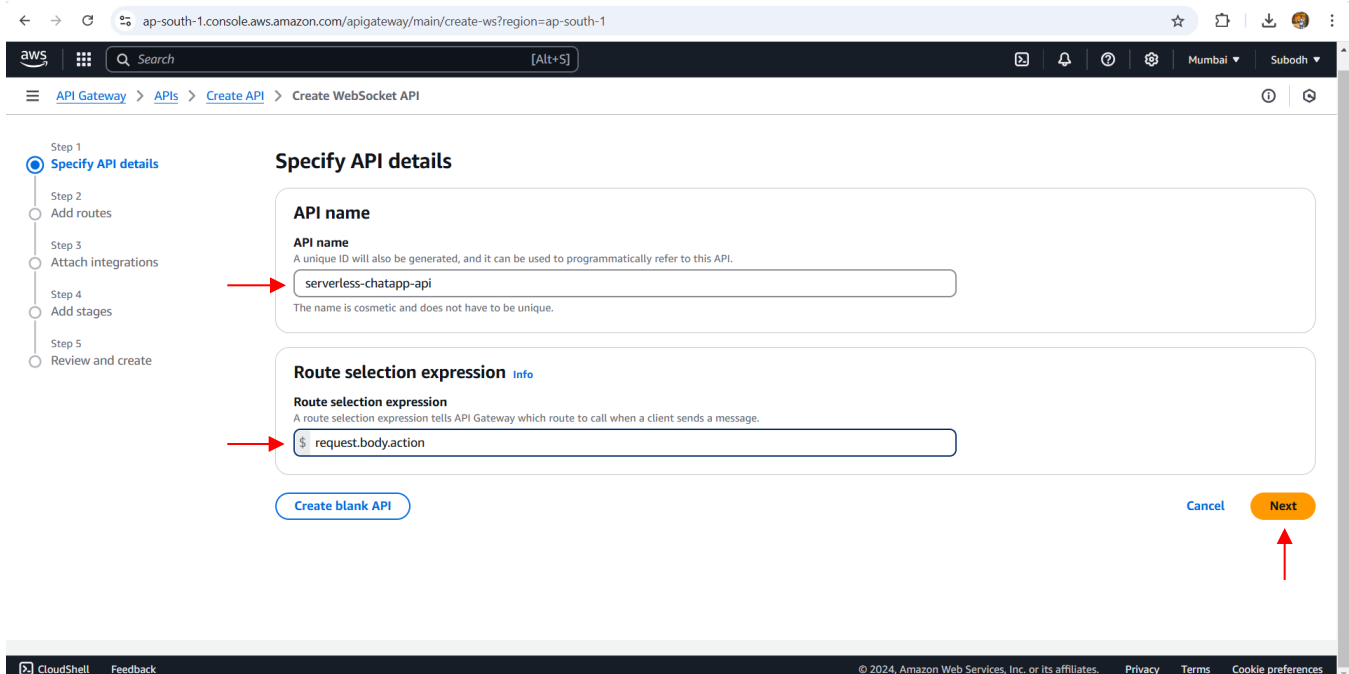
1. Choose "Create API":
 - On the API Gateway homepage, select **"Create API"**.
2. Select WebSocket API:
 - Under **"Choose an API type"**, select **"WebSocket API"**.
 - Click **"Build"** to start building your WebSocket API.

Project by Subodh Kumar



3. Enter API Details:

- **API Name:** Enter serverless-chatapp-api as the name of your API.
- **Route Selection Expression:** Enter request.body.action.



This expression determines how API Gateway routes incoming messages to the appropriate Lambda functions. It looks for an action field in the message body to decide which route to invoke.

4. Choose "Next":

- After entering the API details, click "**Next**" to proceed.

The screenshot shows the AWS API Gateway console in the 'Specify API details' step. The breadcrumb navigation is 'API Gateway > APIs > Create API > Create WebSocket API'. On the left, a sidebar shows the steps: Step 1 (Specify API details), Step 2 (Add routes), Step 3 (Attach integrations), Step 4 (Add stages), and Step 5 (Review and create). The main content area has two sections: 'API name' and 'Route selection expression'. The 'API name' section has a text input field containing 'serverless-chatapp-api'. The 'Route selection expression' section has a text input field containing '\$ request.body.action'. At the bottom, there are three buttons: 'Create blank API', 'Cancel', and 'Next'. Red arrows point to the 'API name' input field, the 'Route selection expression' input field, and the 'Next' button.

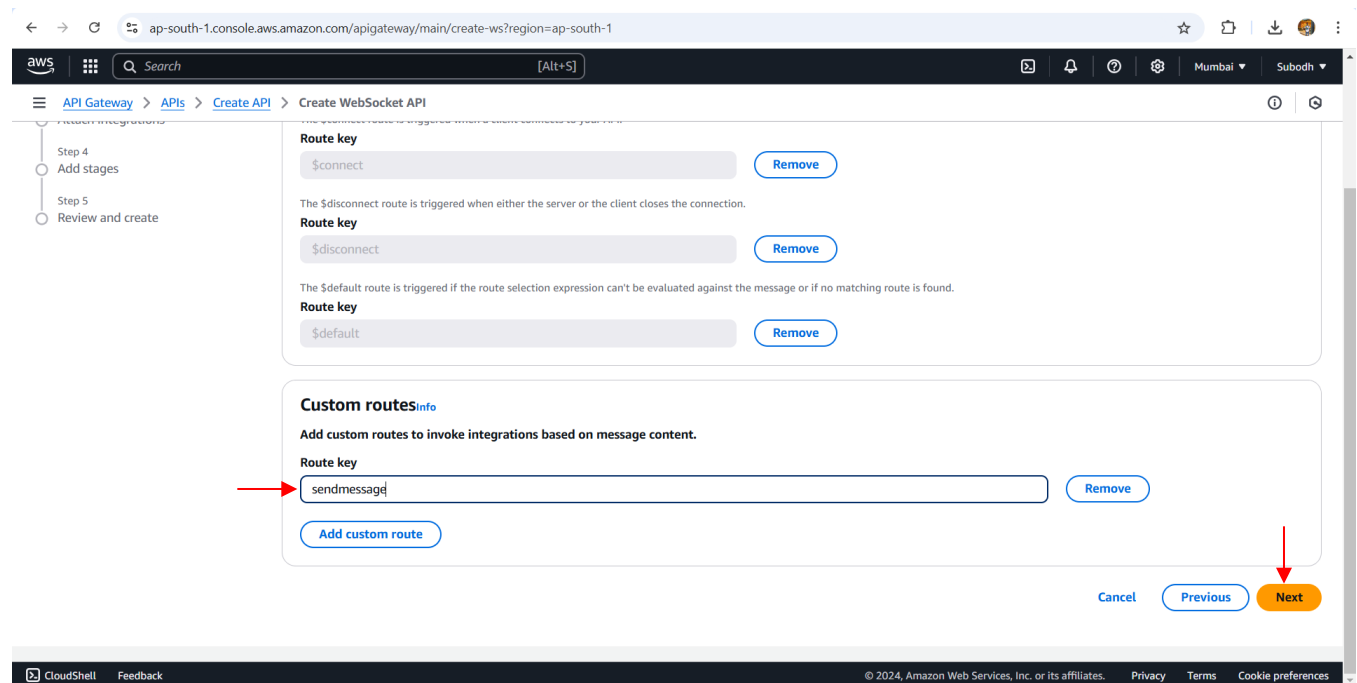
3. Define Routes

1. Add Predefined Routes:

- **\$connect**: This route is automatically triggered when a client connects to the WebSocket API.
- **\$disconnect**: This route is triggered when a client disconnects from the WebSocket API.
- **\$default**: This route is triggered when no other route matches an incoming request.

For Predefined routes, choose Add \$connect, Add \$disconnect, and Add \$default. The \$connect and \$disconnect routes are special routes that API Gateway invokes automatically when a client connects to or disconnects from an API. API Gateway invokes the \$default route when no other routes match a request.

Project by Subodh Kumar



2. Add a Custom Route:

- Click **"Add custom route"**.

3. Route Key: Enter sendmessage.

This route will handle messages sent by clients during a chat session.

4. Review and Continue:

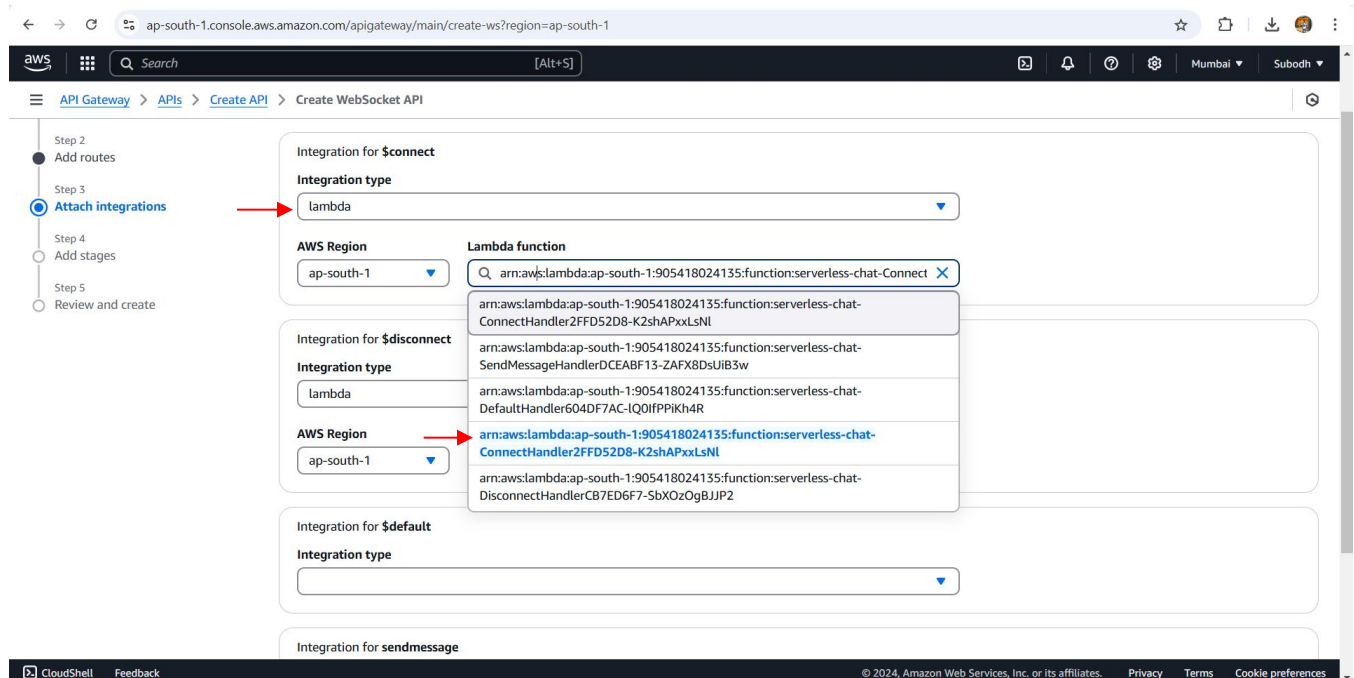
- After adding the predefined and custom routes, review your settings.
- Click **"Next"** to continue.

4. Integrate Routes with Lambda Functions

Once you have created the WebSocket API and defined your routes, the next steps involve integrating these routes with the appropriate Lambda functions that were created earlier. This will allow your API to handle client connections, disconnections, and message transmissions.

Attach Integrations

1. For **Each Route**, Choose Integration Type as Lambda:
2. In the API Gateway console, you'll see the routes you defined earlier (\$connect, \$disconnect, sendmessage, and \$default).



5. Attach Lambda Functions to Routes:

○ **\$connect Route:**

- Set the Integration type to Lambda.
- For Lambda function, choose the function named *serverless-chat-ConnectHandler*.

○ **\$disconnect Route:**

- Set the Integration type to Lambda.
- For Lambda function, choose the function named *serverless-chat-DisconnectHandler*.

○ **sendmessage Route:**

- Set the Integration type to Lambda.
- For Lambda function, choose the function named *serverless-chat-SendMessageHandler*.

○ **\$default Route:**

- You can also attach the default route to a Lambda function or leave it as is for handling unmatched requests.

ap-south-1.console.aws.amazon.com/apigateway/main/create-ws?region=ap-south-1

API Gateway > APIs > Create API > Create WebSocket API

AWS Region: ap-south-1 Lambda function: arn:aws:lambda:ap-south-1:905418024135:function:serverless-chat-Disconn

Integration for \$default

Integration type: lambda

AWS Region: ap-south-1 Lambda function: arn:aws:lambda:ap-south-1:905418024135:function:serverless-chat-Default

Integration for sendMessage

Integration type: lambda

AWS Region: ap-south-1 Lambda function: arn:aws:lambda:ap-south-1:905418024135:function:serverless-chat-SendMe

Cancel Previous Next

1. Review the Attached Integrations:

- Make sure that each route has the correct Lambda function attached. This setup ensures that when a client connects, disconnects, or sends a message, the appropriate Lambda function is triggered.

2. Review the Automatically Created Stage:

- API Gateway automatically creates a deployment stage for you. By default, this stage is named production.
- This stage represents the environment where your API is deployed and can be accessed by users.

ap-south-1.console.aws.amazon.com/apigateway/main/create-ws?region=ap-south-1

API Gateway > APIs > Create API > Create WebSocket API

Step 1: Specify API details
Step 2: Add routes
Step 3: Attach integrations
Step 4: Add stages
Step 5: Review and create

Add stages

Stages

Stage name	Remove
production	Remove

Add stage

Cancel Previous Next

3. Stage Name:

- The stage name production is typical for a live, production-ready environment. You can change this if you prefer (e.g., to dev or test), but for now, using production is fine.

4. Create and Deploy the WebSocket API

- Choose "Next"
- After reviewing the stage, click "Next" to proceed.
- Finally, click "Create and deploy" to deploy your WebSocket API.
- API Gateway will deploy the API, making it available at a specific WebSocket URL.

The screenshot shows the AWS API Gateway console for creating a new WebSocket API. The breadcrumb trail is: API Gateway > APIs > Create API > Create Websocket API. The 'sendmessage' resource is selected. The 'Step 3: Integrations' section shows a table with four integrations, all of type 'Lambda'. The 'Step 4: Stages' section shows a single stage named 'production'. At the bottom right, there are three buttons: 'Cancel', 'Previous', and 'Create and deploy'. A red arrow points to the 'Create and deploy' button.

Route key	Integration type	Integration target
\$connect	Lambda	serverless-chat-ConnectHandler2FFD52D8-K2shAPxxLsNI (ap-south-1)
\$disconnect	Lambda	serverless-chat-DisconnectHandlerCB7ED6F7-SbXOzOgBJJP2 (ap-south-1)
\$default	Lambda	serverless-chat-DefaultHandler604DF7AC-IQ0IFPPIKh4R (ap-south-1)
sendmessage	Lambda	serverless-chat-SendMessageHandlerDCEABF13-ZAFX8DsUiB3w (ap-south-1)

Stage name
production

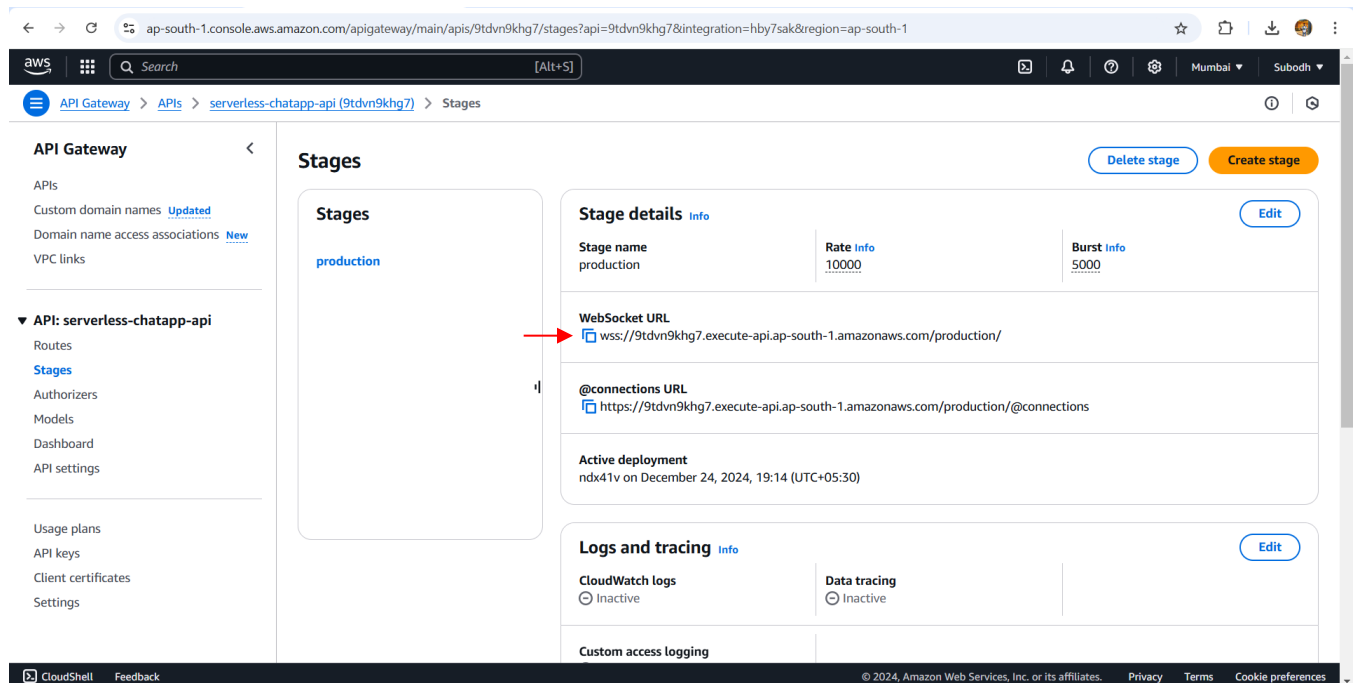
0

Step 3: Test your API

Next, you'll test your API to make sure that it works correctly. Use the `wscat` command to connect to the API.

- To get the invoke URL for your API
- Sign in to the API Gateway console at <https://console.aws.amazon.com/apigateway>.
- Choose your API.
- Choose Stages, and then choose production.

Project by Subodh Kumar



Note your API's **WebSocket URL**. The URL should look like <wss://9tdvn9khg7.execute-api.ap-south-1.amazonaws.com/production/>

To connect to your API

1. Use the following command to connect to your API. for that you can use command prompt.
2. When you connect to your API, API Gateway invokes the `$connect` route. When this route is invoked, it calls a Lambda function that stores your connection ID in DynamoDB.

```
wscat -c wss://9tdvn9khg7.execute-api.ap-south-1.amazonaws.com/production/
```

Open a new command prompt terminal and run the wscat command again with the following parameters.

```
wscat -c wss://9tdvn9khg7.execute-api.ap-south-1.amazonaws.com/production/
```

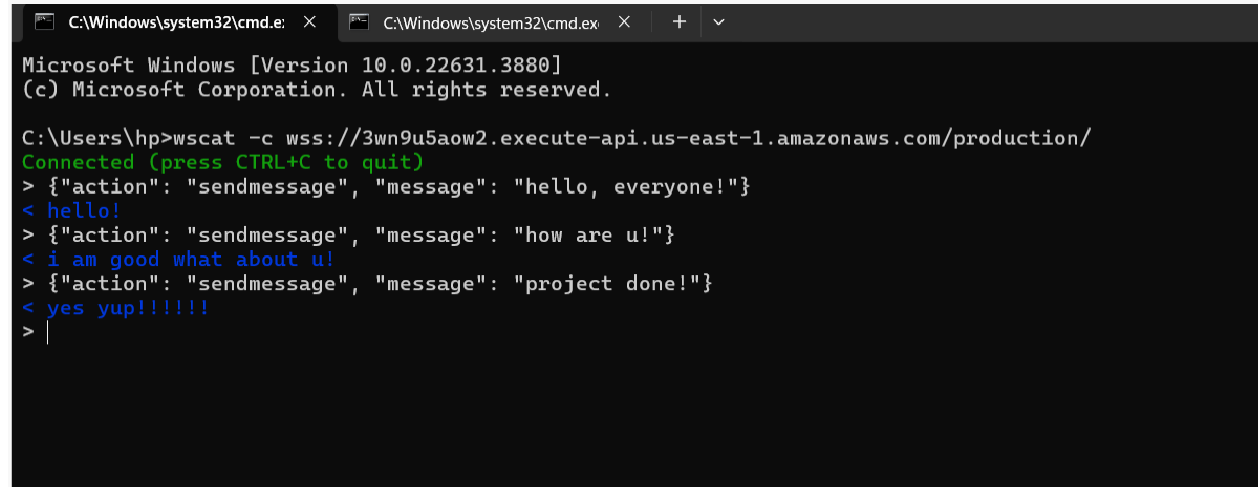
To send a message

- API Gateway determines which route to invoke based on your API's route selection expression. Your API's route selection expression is `$request.body.action`. As a result, API Gateway invokes the `sendMessage` route when you send the following message:
- ```
{"action": "sendMessage", "message": "hello, everyone!"}
```
- The Lambda function associated with the invoked route collects the client IDs from DynamoDB. Then, the function calls the API Gateway Management API and sends the message to those clients. All connected clients receive the following message:

## Project by Subodh Kumar

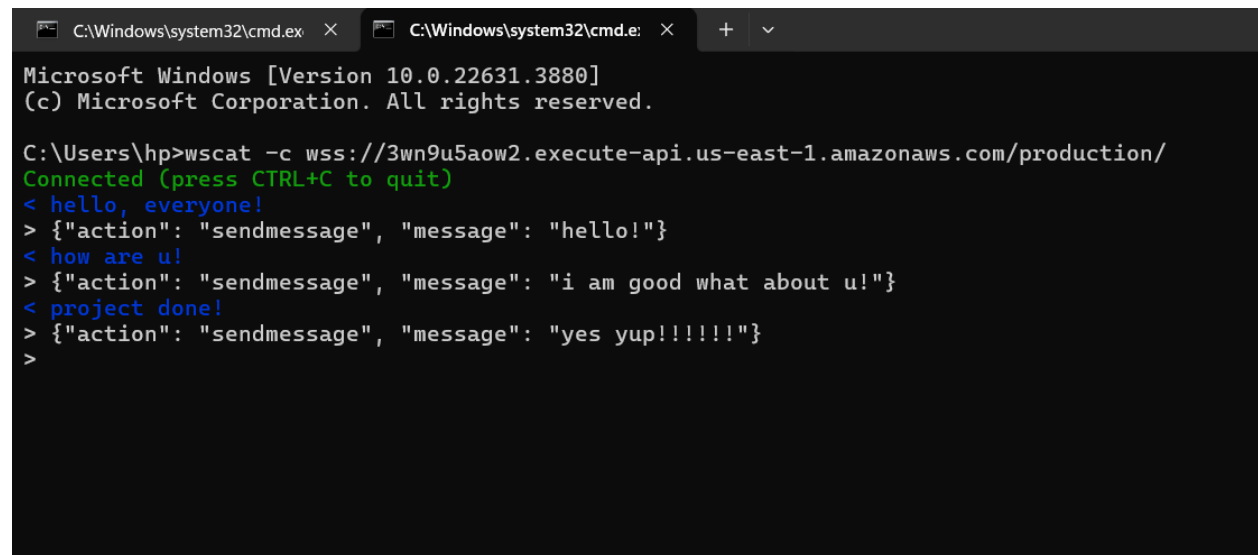
- < hello, everyone!

Connected (press CTRL+C to quit)



```
C:\Windows\system32\cmd.exe X C:\Windows\system32\cmd.exe X + v
Microsoft Windows [Version 10.0.22631.3880]
(c) Microsoft Corporation. All rights reserved.

C:\Users\hp>wscat -c wss://3wn9u5aow2.execute-api.us-east-1.amazonaws.com/production/
Connected (press CTRL+C to quit)
> {"action": "sendmessage", "message": "hello, everyone!"}
< hello!
> {"action": "sendmessage", "message": "how are u!"}
< i am good what about u!
> {"action": "sendmessage", "message": "project done!"}
< yes yup!!!!!!
> |
```



```
C:\Windows\system32\cmd.exe X C:\Windows\system32\cmd.exe X + v
Microsoft Windows [Version 10.0.22631.3880]
(c) Microsoft Corporation. All rights reserved.

C:\Users\hp>wscat -c wss://3wn9u5aow2.execute-api.us-east-1.amazonaws.com/production/
Connected (press CTRL+C to quit)
< hello, everyone!
> {"action": "sendmessage", "message": "hello!"}
< how are u!
> {"action": "sendmessage", "message": "i am good what about u!"}
< project done!
> {"action": "sendmessage", "message": "yes yup!!!!!!"}
>
```

To disconnect from your API

- Press **CTRL+C** to disconnect from your API. When a client disconnects from your API, API Gateway invokes your API's `$disconnect` route. The Lambda integration for your API's `$disconnect` route removes the connection ID from DynamoDB.

Kindly check Lambda function and S3 bucket below.

## Project by Subodh Kumar

The image shows two screenshots of the AWS Management Console. The top screenshot is the AWS Lambda console, displaying a list of functions in the 'ap-south-1' region. The bottom screenshot is the AWS S3 console, displaying a list of buckets in the 'ap-south-1' region.

**AWS Lambda Console (Top):**

- Page title: Functions (4)
- Search bar: Filter by tags and attributes or search by keyword
- Table columns: Function name, Description, Package type, Runtime, Last modified
- Table data:

| Function name                                           | Description | Package type | Runtime      | Last modified  |
|---------------------------------------------------------|-------------|--------------|--------------|----------------|
| serverless-chat-SendMessageHandlerDCEABF13-2AFx8DsUjB3w | -           | Zip          | Node.js 20.x | 29 minutes ago |
| serverless-chat-DefaultHandler604DF7AC-IQOIFPikH4R      | -           | Zip          | Node.js 20.x | 29 minutes ago |
| serverless-chat-ConnectHandler2FFDS2D8-K2shAPxLsNI      | -           | Zip          | Node.js 20.x | 29 minutes ago |

**AWS S3 Console (Bottom):**

- Page title: Buckets
- Section: General purpose buckets (2)
- Search bar: Find buckets by name
- Table columns: Name, AWS Region, IAM Access Analyzer, Creation date
- Table data:

| Name                                     | AWS Region                       | IAM Access Analyzer                          | Creation date                           |
|------------------------------------------|----------------------------------|----------------------------------------------|-----------------------------------------|
| cf-templates-1d87s7jmq2bvk-ap-south-1    | Asia Pacific (Mumbai) ap-south-1 | <a href="#">View analyzer for ap-south-1</a> | December 24, 2024, 18:52:44 (UTC+05:30) |
| elasticbeanstalk-ap-south-1-905418024135 | Asia Pacific (Mumbai) ap-south-1 | <a href="#">View analyzer for ap-south-1</a> | August 17, 2024, 12:04:29 (UTC+05:30)   |

## Step 4: Clean up

To prevent unnecessary costs, delete the resources that you created as part of this tutorial. The following steps delete your AWS CloudFormation stack and WebSocket API.

To delete a WebSocket API

1. Sign in to the API Gateway console at <https://console.aws.amazon.com/apigateway>.

2. On the **APIs** page, select your websocket-chat-app-tutorial API. Choose **Actions**, choose **Delete**, and then confirm your choice.

To delete an AWS CloudFormation stack

1. Open the AWS CloudFormation console at <https://console.aws.amazon.com/cloudformation>.
2. Select your AWS CloudFormation stack.
3. Choose **Delete** and then confirm your choice.

### Credit Attribution :

I'm grateful to [AWS](#) for providing an excellent blog that served as the foundation for this project. This -> [Tutorial: Create a WebSocket chat app with a WebSocket API, Lambda and DynamoDB - Amazon API Gateway](#) was instrumental in guiding the implementation of the base architecture.